

Project Report

Title: Secure Message Transmission Using Asymmetric Encryption

Abstract

In today's digital environment, sensitive information is frequently transmitted across networks that may not be secure. Ensuring confidentiality during communication is a critical requirement for organizations. This project demonstrates the use of asymmetric encryption to securely exchange confidential messages between two users over an untrusted network. Using the RSA-2048 algorithm and OpenSSL tools, a public-private key pair is generated. The sender encrypts the message using the receiver's public key, and only the receiver can decrypt it using the corresponding private key. This approach ensures that confidential information remains protected from unauthorized access during transmission.

1. Introduction

Organizations often need to exchange sensitive information through networks such as the Internet. Since these networks can be monitored or attacked by malicious users, sending information in plain text is highly insecure.

Asymmetric cryptography provides a solution by using two mathematically related keys:

- Public Key – Shared openly and used for encryption.
- Private Key – Kept secret and used for decryption.

In this project, the receiver generates an RSA-2048 key pair and shares his public key with sender. The sender encrypts confidential information using receiver's public key before sending it. Only receiver can decrypt and read the message using his private key.

2. Problem Statement

How can confidential information be transmitted securely over an untrusted network without sharing secret passwords between the communicating parties?

3. Objectives

1. Understand the concept of asymmetric cryptography.
2. Generate RSA public and private keys using OpenSSL.
3. Encrypt a message using a public key.
4. Decrypt the encrypted message using a private key.
5. Demonstrate secure communication over an insecure network.

4. System Requirements

Hardware Requirements

- Computer/Laptop

- Minimum 4 GB RAM
- 10 GB Free Disk Space

Software Requirements

- Operating System: Linux/Windows
- OpenSSL Tool
- Text Editor (Notepad, Nano, Vim, etc.)

5. Methodology

Step 1: Generate RSA Private Key

The Receiver generates a 2048-bit private key:

```
openssl genrsa -out private.pem 2048
```

Step 2: Extract Public Key

The corresponding public key is generated:

```
openssl rsa -in private.pem -pubout -out public.pem
```

Step 3: Create Message File

The sender creates a confidential message:

```
echo "Confidential Company Information" > message.txt
```

Step 4: Encrypt Message Using Public Key

The message is encrypted using Receiver's public key:

```
openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out encrypted.bin
```

Step 5: Transfer Encrypted Message

The encrypted file is transmitted through the network.

Step 6: Decrypt Message Using Private Key

Receiver decrypts the message using his private key:

```
openssl pkeyutl -decrypt -inkey private.pem -in encrypted.bin -out decrypted.txt
```

Step 7: Verify Decrypted Content

```
cat decrypted.txt
```

Output:

Confidential Company Information

7. Results

- RSA-2048 key pair successfully generated.

- Public key shared securely.
- Message encrypted successfully.
- Encrypted data remained unreadable during transmission.
- Original message recovered only by the private key holder.
- Confidentiality maintained throughout communication.

```

kali@kali: ~
Session Actions Edit View Help
└─$ openssl genrsa -out private.pem 2048

(kali@kali)-[~]
└─$ openssl rsa -in private.pem -pubout -out public.pem
writing RSA key

(kali@kali)-[~]
└─$ echo "Confidential Company Information" > message.txt

(kali@kali)-[~]
└─$ openssl pkeyutl -encrypt \ -pubin \ -inkey public.pem \ -in message.txt \ -out encrypted.bin
pkeyutl: Extra option: " -pubin"
pkeyutl: Use -help for summary.

(kali@kali)-[~]
└─$ -help
Command '-help' not found, did you mean:
  command 'dhelp' from deb dhhelp
Try: sudo apt install <deb name>

(kali@kali)-[~]
└─$ help
Command 'help' not found, did you mean:
  command 'yelp' from deb yelp
  command 'dhelp' from deb dhhelp
  command 'helm' from deb kubernetes-helm
Try: sudo apt install <deb name>

(kali@kali)-[~]
└─$ openssl pkeyutl -encrypt \ -pubin \ -inkey public.pem \ -in message.txt \ -out encrypted.bin
pkeyutl: Extra option: " -pubin"
pkeyutl: Use -help for summary.

(kali@kali)-[~]
└─$ openssl pkeyutl -encrypt -pubin -inkey public.pem -in message.txt -out encrypted.bin

(kali@kali)-[~]
└─$ cat decrypted.txt
cat: decrypted.txt: No such file or directory

(kali@kali)-[~]
└─$ openssl pkeyutl -decrypt -inkey private.pem -in encrypted.bin -out decrypted.txt

(kali@kali)-[~]
└─$ cat decrypted.txt
Confidential Company Information

(kali@kali)-[~]
└─$

```

8. Security Analysis

Advantages

- No secret password needs to be shared.
- Secure communication over insecure networks.
- Strong protection against unauthorized access.
- Public key can be distributed freely.

Risks

- Exposure of the private key compromise security.
- RSA operations are computationally heavier than symmetric encryption.
- Proper key management is essential.

9. Applications

- Secure Email Communication
- SSL/TLS Certificates
- Online Banking Systems
- Digital Signatures
- Secure File Transfer
- Government and Corporate Communications

10. Key Learning Outcomes

Through this project, the following concepts were demonstrated:

- Understanding of asymmetric cryptography.
- Usage of RSA public and private key pairs.
- Practical implementation using OpenSSL.
- Secure communication without sharing passwords.
- Importance of protecting private keys.
- Confidentiality preservation in hostile network environments.

11. Conclusion

This project successfully demonstrated secure message transmission using asymmetric encryption. By utilizing RSA-2048 and OpenSSL, confidential information was encrypted with a public key and decrypted using a private key. Even when transmitted through an untrusted network, the data remained protected from unauthorized access. The project highlights the importance of public-key cryptography in modern secure communication systems and reinforces the significance of private key protection.

References

1. OpenSSL Documentation
2. RSA Cryptography Standards (PKCS)
3. Cryptography and Network Security – William Stallings
4. NIST Cryptographic Standards and Guidelines

